OFFSIDE LABS

DLMM 0.10.0

Smart Contract Security Assessment

October 2025

Prepared for:

Meteora

Prepared by:

Offside Labs

Ronny Xing

Sirius Xie

Contents

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

 **https://offside.io/**

 **https://github.com/offsidelabs**

 **https://twitter.com/offside_labs**

2 Executive Summary

Introduction

Offside Labs completed a security audit of *DLMM* smart contracts, starting on May 19th, 2025, and concluding on July 21th, 2025.

Project Overview

Meteora's Dynamic Liquidity Market Maker (DLMM) gives LPs access to dynamic fees and precise liquidity concentration all in real-time. DLMM is designed with features that provide more flexibility for LPs and help LPs earn as much fees as possible with their capital.

DLMM organizes the liquidity of an asset pair into discrete price bins. Token reserves deposited in a liquidity bin are made available for exchange at the price defined for that particular bin. Swaps that happen within the price bin itself would not suffer from slippage. The market for the asset pair is established by aggregating all the discrete liquidity bins.

This update upgrades positions by adding support for dynamic positions and rebalance operations, allowing users to adjust their positions more flexibly. DLMM also adds support for permissionless Token-2022 mints with the Transfer Hook extension.

Audit Scope

The assessment scope contains mainly the smart contracts of the lb-clmm program for the *DLMM* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- DLMM
  - Codebase: `https://github.com/MeteoraAg/DLMM`
  - Release 0.10.0 Commit: 30da0679175a6e3977e2bd4777f4030f05ee2059
  - PR-360
    - Commit Hash: 5b40b411913000747b42222f77a28cdc6b2e7a54
    - Codebase Link: `PR-360`
  - PR-378
    - Commit Hash: b39444d72e458c1239d50929652b0da1187d8914
    - Codebase Link: `PR-378`
  - PR-382
    - Commit Hash: 7fda25a988607f906ffcff4b5a99b1159fef09c5
    - Codebase Link: `PR-382`
  - PR-391
    - Commit Hash: b496f70fe4b5bf0437678e17859dbc61cd804352
    - Codebase Link: `PR-391`

We listed the files we have audited below:

- DLMM
  - programs/lb_clmm/src/*.rs

Findings

The security audit revealed:

- 2 critical issues
- 1 high issue
- 1 medium issue
- 0 low issue
- 7 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	BinArray Account Checks Can Be Bypassed	Critical	Fixed
02	New Bins Reset Can Be Bypassed	Critical	Fixed
03	Lack of Fee Owner Check	High	Fixed
04	DoS due to Redundant Position.is_liquidity_locked Check	Medium	Fixed
05	Missing Update of Position.last_updated_at in Rebalance IX	Informational	Fixed
06	Recommend Always Setting zero_init When Expanding Positions	Informational	Fixed
07	Incorrect Variable Names	Informational	Fixed
08	Incorrect Formula in Comment	Informational	Fixed
09	Certain PDAs May Fail to Add Liquidity via Rebalance Instruction	Informational	Fixed
10	Incorrect Error Types	Informational	Fixed
11	Unreachable Cases in get_position_bin_id_changes Function	Informational	Fixed

4 Key Findings and Recommendations

4.1 BinArray Account Checks Can Be Bypassed

Severity: Critical	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

The `rebalance_liquidity` instruction loads `BinArray` accounts from remaining accounts, and checks if the loading BinArray accounts have the same `lb_pair` pubkey with current context in the function `process_validate_bin_array_and_update_rewards` .

The issue is that, when iterating over the `BinArray` accounts in the remaining_accounts, if rewards exist and the active bin's corresponding BinArray is encountered, subsequent checks on BinArray accounts will be skipped.

```
38      if !is_no_reward && bin_array.update_rewards(&mut lb_pair)? {
39          break;
40      }
```

programs/lb_clmm/src/instructions/rebalance/process_validate_bin_arrays_and_update_rewards.rs#L38-L40

Impact

An attacker could input BinArrays from different lbPairs, then drain the target lbPair's entire liquidity and rewards by exploiting discrepancies in reward/fee rates and liquidity proportions across mixed lbPairs.

Recommendation

Ensure all of the BinArrays in remaining_accounts have been checked.

Mitigation Review Log

Fixed in PR-370

4.2 New Bins Reset Can Be Bypassed

Severity: Critical	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

The rebalance instruction will no longer update the position's `fee_infos` and `reward_infos` before adding liquidity. To prevent newly added bins from being out of sync (which may contain zeros or stale data left by resizing), these bins' `fee_infos` and `reward_infos` must be synchronized prior to liquidity addition.

The issue is that, the function `reset_new_bins` method terminates traversal of the current `new_bin_range` once the bin exceeds the bounds of the current bin array. This results in bins within the second bin array (for a `new_bin_range` spanning two bin arrays) being skipped during reset.

```
773         } else {
774             new_bin_range = new_bin_range_iter.next();
775             break;
776         }
```

programs/lb_clmm/src/state/dynamic_position.rs#L773-L776

Impact

An attacker could use flash loans to deposit a large amount of liquidity into the skipped new bins. During the next fee update, since the `fee_per_token_complete` of these bins remains zero, the fee calculation will amplify exponentially. Ultimately, the attacker can drain the vault by withdrawing the artificially inflated fees.

Recommendation

Recommend flattening the `new_bin_range_iter` into a one dimensional bin ID iterator. Or implementing a triple loop traversal to exhaustively cover each `new_bin_range` without skipping bins.

Mitigation Review Log

Fixed in PR-370

4.3 Lack of Fee Owner Check

Severity: High	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

The instruction `rebalance_liquidity` will claim all of the fee in the range to user's token accounts if the param `should_claim_fee` is true.

But for positions with a different `fee_owner` , fee distributions should be transferred to the fee owner's ATAs, not the user's token accounts.

Impact

Users can steal fees that rightfully belong to the fee owner.

Recommendation

Don't allow to claim fee if the position has a different `fee_owner` .

Mitigation Review Log

Fixed in PR-370

4.4 DoS due to Redundant Position.is_liquidity_locked Check

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

At both the beginning and end of `process_rebalance_remove_liquidity_and_get_amounts_for_removal` , `position.is_liquidity_locked` is called once.

At the beginning and just:

```
30      require!(
31          !position.is_liquidity_locked(Clock::get()?.slot),
32          LBError::LiquidityLocked
33      );
```

programs/lb_clmm/src/instructions/rebalance/process_remove_liquidity.rs#L30-L33

Before the end:

```
103     require!(
104         !position.is_liquidity_locked(pair_type_access_validator.get_
105         current_point()),
106         LBError::LiquidityLocked
107     );
```

programs/lb_clmm/src/instructions/rebalance/process_remove_-liquidity.rs#L103-L107

The purpose of these two checks is the same, so doing it once is sufficient. Additionally, the parameter passed during the first check can only be the current slot, which is incompatible with `ActivationType` being a timestamp in the `lb_pair`.

Impact

The check for `position.is_liquidity_locked` for the current slot at the beginning will cause an error when `lb_pair.pair_type` is `ActivationType::Timestamp`, preventing rebalancing on such `lb_pair` types.

Recommendation

It is recommended to remove the check `!position.is_liquidity_locked(Clock::get()?.slot)` at the beginning of the `process_rebalance_remove_liquidity_and_get_amounts_for_removal` function.

Mitigation Review Log

Fixed in PR-370

4.5 Informational and Undetermined Issues

Missing Update of Position.last_updated_at in Rebalance IX

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

In DLMM, program update `Position.last_updated_at` after users update the liquidity of their position. However, during rebalancing, the position's liquidity is updated but `Position.last_updated_at` is not updated.

It is recommended to also update `Position.last_updated_at` at the end of the rebalancing process.

Recommend Always Setting zero_init When Expanding Positions

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

```
fn realloc_expand_position_account<'info>(
    ...
    position.realloc(new_space, false)?;
```

The `realloc_expand_position_account` function expands the space of the position with a false `zero_init` flag.

Although the current code does not modify the position space twice within the same instruction context, to prevent future code changes from introducing stale data issues, it is recommended to always use `zero_init` when expanding the position space.

Incorrect Variable Names

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

```
184     match (
185         current_add_bin_range.as_mut(),
186         current_remove_bin_range.as_mut(),
187     ) {
```

programs/lb_clmm/src/instructions/rebalance/rebalance_params.rs#L184-L187

Variables are in reverse order. These two variables should be swapped.

Incorrect Formula in Comment

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

```
337     // active_id - (-min_delta_id) = y0 + delta_y * (-min_delta_id)
338     // total_y = y0 + delta_y * (1 + ... + -min_delta_id)
339     // in client, we can fix delta_y = (+-) y0 / (-min_delta_id)
340     // then y0 = total_y / (1 + (-min_delta_id-1)/2)
```

programs/lb_clmm/src/instructions/rebalance/rebalance_params.rs#L337-L340

There are two errors in the above formula:

1. `total_y` should be `y0 * (1 - min_delta_id) + delta_y * (1 + ... + -min_delta_id)`
2. y0 should be `total_y / (1 ± (-min_delta_id + 1)/2)` in the original formula. And should be updated to `total_y / ((- min_delta_id + 1) ± (-min_delta_id + 1) /2)` after fixing the above formula for `total_y` .

Certain PDAs May Fail to Add Liquidity via Rebalance Instruction

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

The `IncreasePositionLength` instruction allows the funder signer to cover additional rent lamports, whereas the `rebalance_liquidity` interface only permits the owner signer to pay rent. Furthermore, it must use a System Program CPI to transfer lamports, this prevents certain PDAs (where the owner is not the System Program) from utilizing the `rebalance_liquidity` interface to add liquidity to new bins.

```
62      transfer(
63          CpiContext::new(
64              system_program,
65              Transfer {
66                  from: owner,
67                  to: position.clone(),
68              },
69          ),
70          extra_lamports,
71      )?;
```

programs/lb_clmm/src/instructions/rebalance/process_resize_position.rs#L62-L71

Incorrect Error Types

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

In `process_rebalance_transfer`, it checks whether the transfer amount meets the slippage requirement. If it doesn't, it throws the error `LBError::ExceededBinSlippageTolerance`.

```
252     require!(
253         excluded_fee_withdraw_amount >= min_withdraw_amount,
254         LBError::ExceededBinSlippageTolerance
255     );
```

programs/lb_clmm/src/instructions/rebalance/rebalance_wrapper.rs#L252-L255

```
271     require!(
272         included_fee_deposit_amount <= max_deposit_amount,
273         LBError::ExceededBinSlippageTolerance
274     );
```

programs/lb_clmm/src/instructions/rebalance/rebalance_wrapper.rs#L271-L274

However, this error type is incorrect; it should be `LBError::ExceededAmountSlippageTolerance`.

Fixed in PR-388

Unreachable Cases in get_position_bin_id_changes Function

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

In the match pattern of the function `get_position_bin_id_changes`, `non_zero_liquidity_max_id` and `non_zero_liquidity_min_id` must be either both `None` or

both `Some` . Consequently, the following four scenarios are impossible.

```
        (Some(min_id), None, None) => (min_id, min_id),
        (None, Some(max_id), None) => (max_id, max_id),
        (None, Some(max_id), Some((min_add_id, max_add_id))) => {
            (min_add_id, max_add_id.max(max_id))
        }
        (Some(min_id), None, Some((min_add_id, max_add_id))) => {
            (min_id.min(min_add_id), max_add_id)
        }
```

programs/lb_clmm/src/instructions/rebalance/process_resize_position.rs#L32-L39

We recommend explicitly throwing errors for these cases to prevent future modifications from introducing undetected logical errors:

```
    (Some(_), None, _) | (None, Some(_), _) => unreachable!() //
↪   unreachable here is not recommended
```

Fixed in PR-388

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

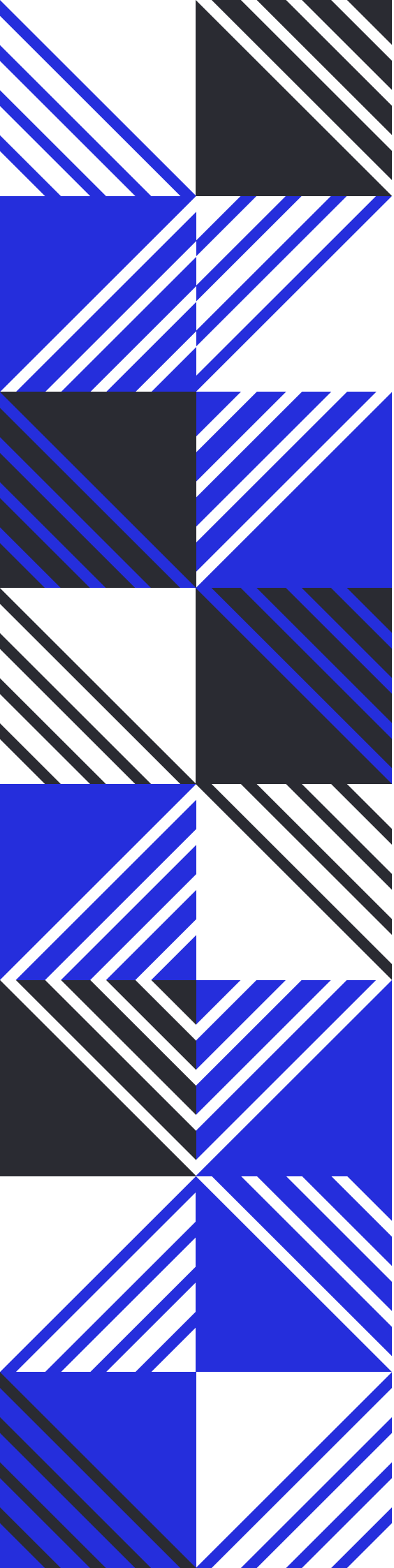
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs